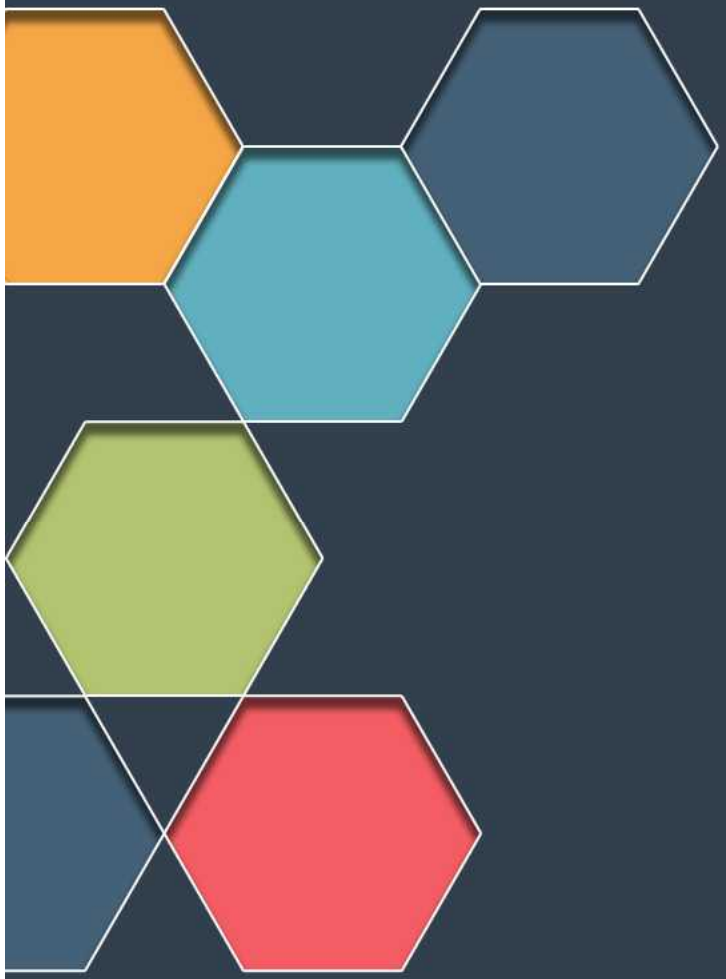


제5장 지식표현

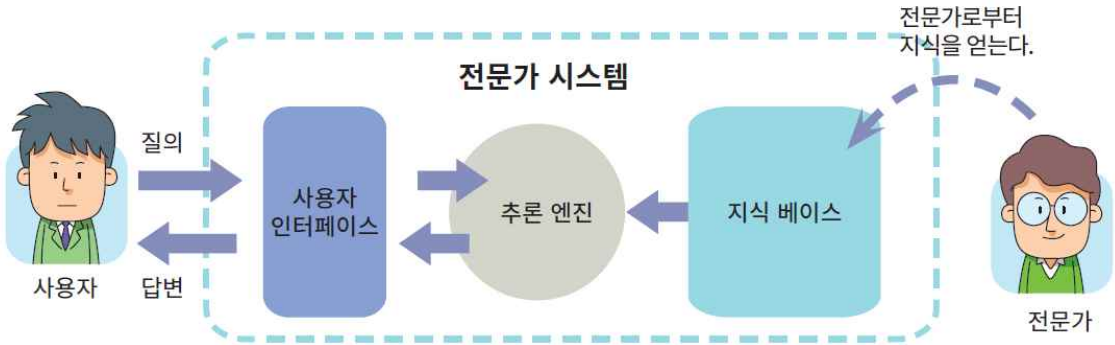
학습 목표

- 게임 속 사실과 규칙을 명제논리로 표현한다.
- 진리표와 논리융합으로 결론을 검증한다.
- 술어·변수·한정사로 일반 규칙을 기술한다.
- 단일화, CNF 변환, Prolog 질의를 수행한다.

규칙

- 규칙은 전제(IF문)와 결론(THEN)의 두 부분으로 구성된다. 규칙의 전제 조건이 일치하는 경우, 규칙은 점화되고 결론 부분이 실행된다.
- 규칙을 사용하는 시스템에는 작업 메모리라고도 하는 데이터베이스가 포함된다. 작업 메모리에는 현재 관측된 사실이나 상태, 지식이 저장된다

규칙 #1: IF 비가 온다.
THEN 우산을 가져간다.
규칙 #2: IF 프로그램에 버그가 없다.
THEN 프로그램은 올바르게 동작한다.
규칙 #3: IF 습도가 높다 OR 온도가 30도 이상이다.
THEN 에어콘을 가동한다.



게임 AI에서 논리의 역할

사실, 규칙, 목표를 구별한다.

사실: Sees(guard, hero)

규칙: Sees(g,h) → Alert(g)

목표: Alert(guard)를 증명할 수 있는가?

실시간 관측값이 바뀌면 사실 집합도 갱신한다.

06 명제 논리

- 기호 논리학에서 명제(proposition)는 참(true, T)이거나 또는 거짓(false, F)을 판별할 수 있는 문장이다.

P = 마트는 월요일부터 토요일까지 영업한다.
Q = 오늘은 일요일이다.
R = 오늘 마트는 영업하지 않는다.

- 논리 연산자 사용 가능: 단순 명제들을 결합해 복합 명제를 만듦

Q = 오늘은 일요일이다.
NOT Q = 오늘은 일요일이 아니다.

J = 옷은 파랑색이다.
K = 옷은 스트라이프 무늬가 있다.
L = J AND K = 옷은 파랑색이고 스트라이프 무늬가 있다.

질문 | 명제 만들기

잠입 게임의 경비 규칙을 기호로 쓰자.

S: 소음이 들림

A: 경비가 경계 상태임

"소음이 들리면 경계한다"의 논리식은?

소음이 없을 때도 경계할 수 있을까?

풀이 :

명제논리에서의 추론

- 추론(inference)이란 우리가 가지고 있는 지식(규칙)과 우리가 이미 알고 있는 사실로부터 새로운 사실을 유추해 내는 것이다.

(지식) 우리집 강아지는 집안에 있거나 앞마당에 있다.
(사실) 강아지가 집안에 없다.

==========
(추론된 사실) 따라서 강아지는 앞마당에 있을 것이다.

추론법칙

- 모더스 포넌스(Modus Ponens)

규칙 | A → B
사실 | A

결론 B

"홍길동이 세계 일주 중이라면 → 로또에 당첨된 것이다. “
"홍길동은 세계 일주 중이다."

∴ ”홍길동은 로또에 당첨된 것이다. “

추론법칙

- 부정 논법(Modus Tollens)

규칙 | A → B
사실 | NOT B

결론 NOT A

"어떤 동물이 강아지라면 → 어떤 동물은 4개의 다리를 가지고 있다."
"어떤 동물은 4개의 다리를 가지고 있지 않다."

∴ "어떤 동물은 강아지가 아니다."

추론법칙

- 삼단논법(syllogism)

규칙 | A → B
사실 | B → C

결론 A → C

"경비가 적을 보면 경계한다"
"경계하면 동료에게 알린다"

∴ "적을 보면 동료에게 알린다"

명제 논리

❖ **명제 논리 (propositional logic)**

- **명제**(命題, proposition)
  - 참, 거짓을 분명하게 판정할 수 있는 문장
    - 아리스토텔레스는 플라톤의 제자이다. (명제)
    - 1+1 = 3. (명제)
    - 일어나서 아침 먹자. (명제 아님)
- 명제를 **P, Q**등과 같은 **기호로 표현**
- 명제 기호의 **진리값**(truth value)을 사용하여 명제들에 의해 표현되는 문장들의 진리값 결정
- 문장 자체의 **내용**에 대해서는 **무관심**, 문장의 **진리값**에만 **관심**

명제 논리

- **기본 명제**(primitive proposition)
  - 하나의 진술(statement)로 이루어진 최소 단위의 명제
- **복합 명제**(compound proposition)
  - 기본 명제들이 결합되어 만들어진 명제
- 예.
  - 알렉산더는 아시아를 넘본다 ⇨ P
  - 징기스칸은 유럽을 넘본다 ⇨ Q
  - 알렉산더는 아시아를 넘보고, 징기스칸은 유럽을 넘본다 ⇨ $P \wedge Q$

명제 논리의 구문

❖ **논리식**(logical expression)

- 명제를 기호로 표현한 형식
- **명제기호**, 참과 거짓을 나타내는 **T**와 **F**, 명제 기호를 연결하는 **논리기호**인 ¬, ∨, ∧, →, ≡를 사용하여 구성

표 3.1 논리기호

논리기호	이름	논리식	의미
¬	부정(negation)	$\neg P$	P 가 아님
∨	논리합(disjunction)	$P \vee Q$	P 또는 Q
∧	논리곱(conjunction)	$P \wedge Q$	P 그리고 Q
→	함의(implication)	$P \rightarrow Q$	P 이면 Q
≡	동치(equivalence)	$P \equiv Q$	$(P \rightarrow Q) \wedge (Q \rightarrow P)$

명제 논리의 구문

- **리터럴**(literal)
  - 명제 기호 P와 명제 기호의 부정 $\neg P$
- **절**(clause)
  - 리터럴들이 논리합(∨)으로 연결된 논리식

$$P \vee Q \vee \neg R \quad (\text{논리합 절})$$

$$P \wedge Q \wedge \neg R \quad (\text{논리곱 절})$$

- **논리곱 정규형 (conjunctive normal form, CNF)**
  - 논리합 절들이 논리곱으로 연결되어 있는 논리식

$$(P \vee Q \vee \neg R) \wedge (\neg Q \vee R \vee S) \wedge (P \vee R \vee S)$$

- **논리합 정규형 (disjunctive normal form, DNF)**
  - 논리곱 항(term)들이 논리합으로 연결된 논리식

$$(P \wedge Q \wedge \neg R) \vee (\neg Q \wedge R \wedge S) \vee (P \wedge R \wedge S)$$

명제 논리의 구문

❖ **정형식**(well-formed formula, **wff**)

- 논리에서 문법에 맞는 논리식
- **명제 논리에 대한 정형식**

(1) 진리값 T, F와 명제 기호들 $P, Q, R, \cdots$은 정형식이다.

(2) p와 q가 정형식이면, 논리 기호를 사용하여 구성되는 논리식 $\neg p$, $p \vee q$, $p \wedge q$, $p \rightarrow q$, $p \equiv q$도 정형식이다.

(3) (1)과 (2)에 의해 정의되는 논리식만 정형식이다.

$$(P \wedge Q) \rightarrow \neg P$$

$$P \rightarrow \neg P$$

$$P \vee P \rightarrow P$$

$$(P \rightarrow Q) \rightarrow (\neg Q \rightarrow \neg P)$$

명제 논리의 의미

❖ **진리표**(truth table)

- 논리기호에 따라 참, 거짓 값을 결합하는 방법을 나타낸 표

표 3.2 진리표

P	Q	$\neg P$	$P \vee Q$	$P \wedge Q$	$P \rightarrow Q$	$P \equiv Q$
F	F	T	F	F	T	T
F	T	T	T	F	T	F
T	F	F	T	F	F	F
T	T	F	T	T	T	T

❖ 논리식의 **해석**(interpretation)

- **논리식**의 **진리값**을 **결정**하는 것
  - P = T, Q = F, R = T

    $(P \vee \neg Q) \wedge (Q \vee \neg R)$ = F
- 우선 각 **명제기호**의 **진리값** 결정 필요
  - 명제 기호에 "명제"를 대응시키고, 해당 명제의 진리값을 결정

    예. $\boldsymbol{P}$ => "**토마토는 과일이다**", P = F
  - **대응된 명제**를 명제 기호의 **외연**(外延) 또는 **의미**(denotation)라 함

명제 논리의 의미

❖ 논리식의 **모델**(model)

- 논리식의 **명제기호**에 **참값**(T) 또는 **거짓값**(F)을 **할당**한 것
  - $(P \vee \neg Q) \wedge (Q \rightarrow \neg R)$: $P = \mathrm{T},\ Q = \mathrm{F},\ R = \mathrm{T}$
- 모델이 주어지면, **진리표**를 사용하여 논리식의 진리값 결정, 즉 **해석** 가능

P	Q	$\neg P$	$P \vee Q$	$P \wedge Q$	$P \rightarrow Q$	$P \equiv Q$
F	F	T	F	F	T	T
F	T	T	T	F	T	F
T	F	F	T	F	F	F
T	T	F	T	T	T	T

- n**개**의 **명제기호**가 **논리식**에 사용된다면, 각각 T 또는 F값을 가질 수 있기 때문에, 총 2^n**개**의 **모델**이 존재

질문 | 필요조건과 충분조건

“열쇠가 있어야 문이 열린다”를 해석하자.

K: 열쇠 보유, O: 문 열림

① 원문을 K→O와 O→K 중에서 고른다.

② O가 참이고 K가 거짓이면 규칙은?

③ 열쇠만 있으면 문이 열린다는 뜻인가?

풀이 :

명제 논리의 의미

- **타당한 논리식** (**valid** logical expression)
  - 모든 가능한 모델에 대해서 **항상 참**(T)인 논리식
  - **항진식**(恒眞式, tautology)
    - 예. $P \vee \neg P$

      P = T인 경우 : $P \vee \neg P$ = T

      P = F인 경우 : $P \vee \neg P$ = T
- **항위식**(恒僞式, contradiction)
  - 모든 가능한 모델에 대해서 **항상 거짓**이 되는 논리식
    - 예. $P \wedge \neg P$

      P = T인 경우 : $P \wedge \neg P$ = F

      P = F인 경우 : $P \wedge \neg P$ = F

명제 논리의 의미

❖ **충족가능한**(satisfiable) 논리식
- 참으로 만들 수 있는 모델이 하나라도 있는 논리식
  - 예. $(P \vee \neg Q) \wedge (Q \vee \neg R)$

    P = T, Q = T, R = F

❖ **충족불가능한**(unsatisfiable) 논리식
- 참으로 만들 수 있는 모델이 전혀 없는 논리식
- **항위식**인 논리식
  - 예. $P \wedge \neg P$

명제 논리의 의미

❖ **동치관계**(equivalence relation)의 논리식

- **어떠한 모델**에 대해서도 **같은 값**을 갖는 두 논리식

(1) $\neg(\neg p) \equiv p$

(2) $p \vee \mathrm{F} \equiv p,\ \ p \wedge \mathrm{T} \equiv p$

(3) $p \vee \neg p \equiv \mathrm{T},\ \ p \wedge \neg p \equiv \mathrm{F}$

(4) $\neg(p \wedge q) \equiv \neg p \vee \neg q,\ \neg(p \vee q) \equiv \neg p \wedge \neg q$

(5) $p \rightarrow q \equiv \neg p \vee q$

(6) $p \vee (q \vee r) \equiv (p \vee q) \vee r,\ p \wedge (q \wedge r) \equiv (p \wedge q) \wedge r$

(7) $p \vee (q \wedge q) \equiv (p \vee q) \wedge (p \vee q),\ p \wedge (q \vee q) \equiv (p \wedge q) \vee (p \wedge q)$

임의의 정형식을 절의 논리곱으로 변환하기

- **명제논리의 임의의 정형식은 이와 동치인 절의 논리곱으로 변환 가능**
- **정규형**
  - 논리곱 정규형(conjunctive normal form, CNF): 절(논리합)의 논리곱으로 나타낸 정형식
  - 논리합 정규형(disjunctive normal form, DNF): 논리곱 절의 논리합으로 나타낸 정형식

임의의 정형식을 CNF로 변환하는 절차

예제) ¬(P⊃Q)∨(R⊃P)

1. 함의 기호(⊃)를 ∨를 이용한 동치식으로 변환: ¬(¬P∨Q)∨(¬R∨P)
2. 이중 ¬ 기호 삭제 (드모르간 법칙): (P∧¬Q)∨(¬R∨P)
3. 결합법칙과 배분법칙을 이용하여 CNF로 변환:
   (P∨¬R∨P)∧(¬Q∨¬R∨P)
   (P∨¬R)∧(¬Q∨¬R∨P)

- 절의 논리곱(CNF 형태)은 절의 집합으로 표시: {(P∨¬R),(¬Q∨¬R∨P)}

질문 | CNF 변환

협동 퍼즐의 문 열림 규칙을 변환하자.

K: 열쇠, S: 스위치, O: 문 열림

$(K \wedge S) \rightarrow O$

① 함의를 제거한다.

② 드모르간 법칙으로 절을 만든다.

풀이 :

귀결 (entailment)

- 집합 Δ에 있는 모든 wff를 True로 하는 해석에 대해 wff ω가 True값을 가질 때
  - Δ⊨ω로 표시
  - Δ는 ω를 **논리적으로 귀결한다**(logically entails)
  - ω는 Δ를 **논리적으로 따른다**(logically follows)
  - ω는 Δ의 **논리적 결론**(logical consequence)

    예) {P} ⊨ P, {P, P⊃Q} ⊨ Q, F ⊨ ω (ω는 임의의 wff), P∧Q ⊨ P

명제 논리의 의미

❖ **논리적 귀결**(logical entailment)

- Δ : 정형식(wff)의 집합. $\Delta = \{P, P \rightarrow Q\}$
  ω : 정형식. $\omega = Q$
- Δ에 있는 모든 정형식을 **참**(T)으로 만드는 **모델**이, ω를 **참**(T)으로 만든다
  = Δ는 ω를 **논리적으로 귀결**한다(**logically entail**)
  = ω는 Δ를 **논리적으로 따른다**(**logically follow**)
  = ω는 Δ의 **논리적 결론**(**logical consequence**)이다
- **표기법 :** $\Delta \models \omega$
  - $\{P, P \rightarrow Q\} \models Q$

P	Q	$\neg P$	$P \vee Q$	$P \wedge Q$	$P \rightarrow Q$	$P \equiv Q$
F	F	T	F	F	T	T
F	T	T	T	F	T	F
T	F	F	T	F	F	F
T	T	F	T	T	T	T

  - Δ가 참이면, ω도 참이다
- **추론**
  - 참으로 알려진 Δ로부터, 알려지지 않은 참인 ω를 찾는 것

명제 논리의 추론

- **추론**(推論, inference)
  - **귀납적 추론**(inductive inference)
    - 관측된 복수의 **사실들**을 **일반화**(generalization)하여 **일반적인 패턴** 또는 **명제**를 도출하는 것
  - **연역적 추론**(deductive inference)
    - **참**인 **사실들** 또는 **명제들**로부터 **새로운** 참인 **사실** 또는 **명제**를 도출하는 것
  - **논리**에서의 **추론**
    - 연역적 추론의 형식화된 버전
    - 함의(→)의 논리적 관계를 이용하여 **새로운 논리식**을 **유도**해 내는 것
    - 함의 $\boldsymbol{p \rightarrow q}$
      - $\boldsymbol{p}$ **: 전제**(premise)
      - $\boldsymbol{q}$ **: 결론**(conclusion, consequence)
      - p가 참일 때, q가 참임

명제 논리의 추론

❖ **추론규칙**(inference rule)

- 참인 논리식들이 **논리적으로 귀결**하는 **새로운 논리식**을 만들어내는 **기계적**으로 **적용**되는 규칙

추론 표기법: $\vdash$

- **긍정 논법** (modus ponens)

$$\frac{p \rightarrow q \quad p}{q}$$

새이다 → 날 수 있다
새이다

날 수 있다

$p \rightarrow q, p \vdash q$

- **부정 논법** (modus tollens)

$$\frac{p \rightarrow q \quad \neg q}{\neg p}$$

새이다 → 날 수 있다
날 수 없다

새가 아니다

$p \rightarrow q, \neg q \vdash \neg p$

- **삼단 논법** (syllogism)

$$\frac{p \rightarrow q \quad q \rightarrow r}{p \rightarrow r}$$

새이다 → 날개가 있다
날개가 있다 → 날 수 있다

새이다 → 날 수 있다

$p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$

명제 논리의 추론

❖ **추론규칙** – cont.

- **논리 융합 (resolution)**
  - 일반화된 추론규칙
    - 긍정 논법, 부정 논법, 삼단 논법의 규칙을 포함한 추론 규칙
  - **두 개**의 **논리합절**이 **같은 기호**의 **긍정**과 **부정**의 **리터럴**을 서로 포함하고 있을 때, 해당 리터럴들을 제외한 **나머지 리터럴들**의 **논리합절**을 만들어 내는 것

$$P \vee q,\ \neg P \vee r \ \vdash\ q \vee r$$

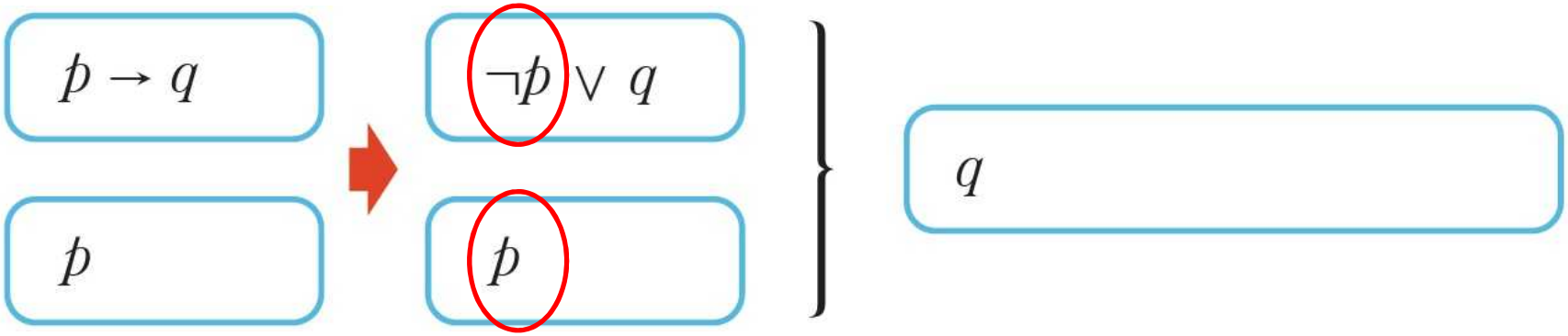
논리융합식(resolvent)

논리융합

❖ 논리융합은 같은 기호의 리터럴과 부정 리터럴을 가지고 있는 2개의 절을 조합하여서 새로운 절을 생성하는 방법이다.



논리융합으로 모더스 포넌스를 유도해보자.



질문 | 논리융합 한 단계

두 절에서 새로운 절을 만들어 보자.

¬Seen ∨ Alert

Seen ∨ Hide

상보 리터럴 한 쌍을 골라 융합하면?

결과가 Alert와 Hide 중 하나를 보장하는가?

풀이 :

명제 논리의 추론

❖ **추론 규칙의 정당성과 완전성**

- **추론 규칙의 정당성 (sound)**
  - 추론 규칙에 의해 생성된 논리식은 주어진 논리식들이 논리적으로 귀결하는 것이다.
  - 즉, 추론 규칙이 만들어 낸 것은 항상 참이다.

$$\Delta \vdash \omega \quad \rightarrow \quad \Delta \models \omega$$

- **추론 규칙의 완전성 (complete)**
  - 주어진 논리식들이 **논리적으로 귀결하는 것**들은 **추론 규칙**이 **찾아 낼 수 있다**.

명제 논리의 추론

- **정리증명**(theorem proving)
  - **공리**(axiom)
    - 추론을 할 때, 참인 것으로 주어지는 논리식
  - **정리**(theorem)
    - 공리들에 추론 규칙을 적용하여 얻어지는 논리식
  - **정리 증명**
    - 공리들을 사용하여 정리가 참인 것을 보이는 것
    - **구성적 증명**(**constructive proof**)
      - 공리들에 추론 규칙들을 적용하여 증명을 만들어 보이는 증명
    - **논리융합 반박**(**resolution refutation**)
      - 증명할 정리를 부정(negation)한 다음, 논리융합 방법을 적용하여 모순이 발생하는 것을 보여서, 정리가 참임을 증명하는 방법

절에서의 논리융합

❖ Σ_1, Σ_2: 리터럴의 집합, λ: 아톰
- $\{\lambda\} \cup \Sigma_1$과 $\{\neg\lambda\} \cup \Sigma_2$로부터 $\Sigma_1 \cup \Sigma_2$를 추론
- $\Sigma_1 \cup \Sigma_2$를 두 절의 **논리융합식**(resolvent)이라 부름

예)
- R∨P와 ¬P∨Q는 R∨Q로 논리융합됨: 연결(chaining)
- R과 ¬R∨P는 P로 논리융합됨: **모더스 포넌스**
- P∨Q∨R∨S와 ¬P∨Q∨W는 Q∨R∨S∨W로 논리융합됨
- P∨Q∨¬R과 P∨W∨¬Q∨R의 경우
  - Q에 대해 논리융합: P∨¬R∨R∨W -> T
  - R에 대해 논리융합: P∨Q∨¬Q∨W -> T
- λ와 ¬λ를 논리융합하면 공절(empty clause): F를 추론

논리융합의 정당성(soundness)

- {λ}∪Σ_1과 {¬λ}∪Σ_2가 둘 다 True값을 가지면 논리융합식 $\Sigma_1 \cup \Sigma_2$도 True

  (증명) "사례별로 추론"
  - (사례 1) λ가 True → {¬λ}∪Σ_2절이 True이기 위해서는 Σ_2가 True
  - (사례 2) λ가 False => {λ}∪Σ_1절이 True이기 위해서는 Σ_1이 True
  - 따라서 $\Sigma_1 \cup \Sigma_2$는 True

논리융합 반박 (Resolution refutation)

❖ **논리융합 반박의 필요성**

- 직접 유도는 완전하지 않지만, 반박 증명은 완전하다.

  예) $P \wedge R \models P \vee R$, 하지만 {P, R}로부터 논리융합 방법으로 $P \vee R$ 추론 불가능

- $\neg(P \vee R)$을 {P,R}에 추가하면 공절을 도출할 수 있다.
- 명제논리 절의 논리융합은 반박 증명에서 완전하다.

논리융합 반박

- 모순을 이용한 논리융합 (목표의 부정 -> 모순 유도 -> 목표 증명)

  예) P∨R의 부정: {¬P, ¬R} → {P, R}과 논리융합하면 공절이 만들어짐

- wff의 집합 Δ로부터 임의의 wff ω를 증명하기 위한 논리융합 반박 절차

  1. Δ에 있는 wff들을 절 형태(clause form)로 변환(CNF)
  2. 증명하고자 하는 ω의 **부정**을 절 형태로 변환
  3. 1과 2단계의 결과인 절을 하나의 집합 Γ으로 합침
  4. Γ에 있는 절에 논리융합을 반복 적용: 공절이 나올 때까지 (또는 더 이상 Γ에 새로운 절이 추가되지 않을 때까지)

질문 | 규칙 충돌 찾기

보스 방 경보 규칙에 모순이 있는가?

D→A, W→¬A, D, W

D: 문 잠김, W: 창문 열림, A: 경보 작동

어떤 사실과 절을 융합해 공절을 만들까?

게임 규칙을 어떻게 고칠까?

풀이 :

명제 논리의 추론

❖ 논리융합 반박을 이용한 정리증명의 예

- 공리 (axiom)

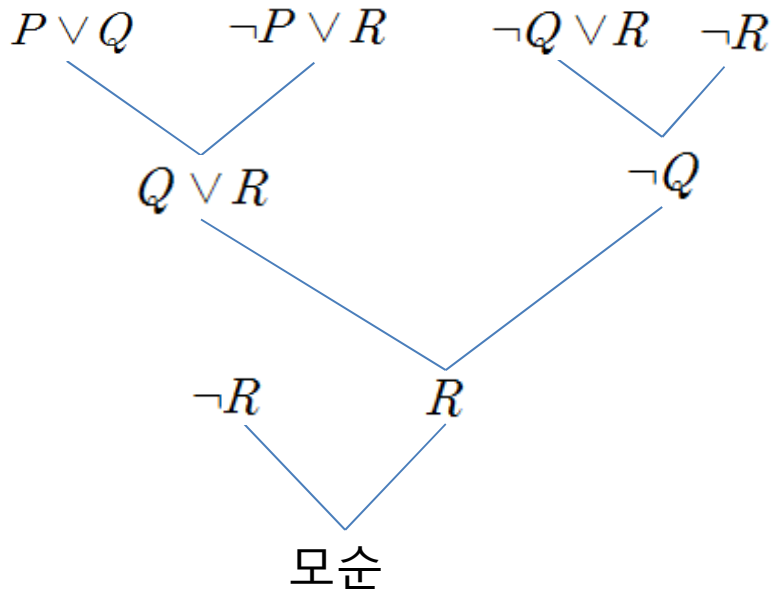
$P \vee Q$

$P \rightarrow R \qquad \neg P \vee R$

$Q \rightarrow R \qquad \neg Q \vee R$

- 정리 (theorem)

$R \quad \Rightarrow \quad \neg R$



명제 논리의 지식표현

❖ **명제 논리를 이용한 지식 표현**
- 문장으로 표현된 지식으로부터 **기본 명제**들을 **추출**
- 각 명제에 대해 **명제기호 부여**
- 기본 명제들의 **논리적 연결 관계**를 참고하여 대응되는 명제 기호들을 **논리기호**로 **연결**하여 논리식 구성

❖ **명제 논리로 표현된 지식에 대한 추론**
- **명제 기호**가 나타내는 명제의 **의미**와는 **무관**
- 대수적인 **기호 연산**을 통해서 **추론** 수행

❖ 앞면이면 내가 이기고, 뒷면이면 네가 진다. 이 문장을 명제논리로 표현하고 내가 이긴다는 것을 논리융합을 이용하여 증명하라. 필요하면 다른 문장을 만들어 사용해도 좋다.

1.	H ⊃ IW	1. ¬H ∨ IW
2.	Tail ⊃ YL	2. ¬Tail ∨ YL
3.	H ∨ Tail	3. H ∨ Tail
4.	YL ⊃ IW	4. ¬YL ∨ IW
5.	¬IW	5. ¬IW
6.	H ∨ YL	: 2와 3
7.	YL ∨ IW	: 6과 1
8.	IW	: 7과 4
9.	Nil	: 8과 5

❖ 다음 명제논리 wff를 CNF 형태로 바꾼 다음 절의 집합으로 표기하라.

$\neg[((P \vee \neg Q) \supset R) \supset (P \wedge R)]$

$\equiv \neg[\neg((P \vee \neg Q) \supset R) \vee (P \wedge R)]$

$\equiv \neg[\neg(\neg(P \vee \neg Q) \vee R) \vee (P \wedge R)]$

$\equiv (\neg(P \vee \neg Q) \vee R) \wedge \neg (P \wedge R)$

$\equiv ((\neg P \wedge Q) \vee R) \wedge (\neg P \vee \neg R)$

$\equiv ((\neg P \vee R) \wedge (Q \vee R)) \wedge (\neg P \vee \neg R)$

$\equiv ((\neg P \vee (R \wedge \neg R)) \wedge (Q \vee R))$

$\equiv (\neg P) \wedge (Q \vee R)$

$\{\neg P, Q \vee R\}$

07 술어논리

- 명제 논리에서 하나의 명제가 나누어질 수 없기 때문에 어려움이 있다. 즉 우리는 전체 명제가 참이냐 거짓이냐 만을 말할 수 있다.
  - 예를 들어서 “신호등이 파랑색이다.”라는 명제가 있다면 이것을 “신호등”과 “파랑색이다”로 나눌 수 있다면 아주 편리할 것이다.
- 술어 논리에서는 하나의 명제가 객체(object, 또는 인수)와 술어(predicate)로 나누어진다.
- 변수와 한정사를 사용할 수 있다.
- 이와 같은 술어 논리의 특징 때문에 명제를 사용하는 것보다, 훨씬 더 구체적으로 지식을 표현할 수 있다.

술어 논리

❖ **술어 논리**(predicate logic)

- 명제의 내용을 다루기 위해 **변수**, **함수** 등을 도입하고 이들의 값에 따라 참, 거짓이 결정되도록 명제 논리를 확장한 논리

- **술어**(述語, predicate)
  - 문장의 '주어+서술어'형태에서 **서술어**에 해당
  - 대상의 **속성**이나 **대상 간의 관계**를 기술하는 기호
  - **참**(T) 또는 **거짓**(F) **값**을 갖는 **함수**
  - 예.
    - **Student**(John)
    - **Friend**(John, Mary)

술어 논리의 예

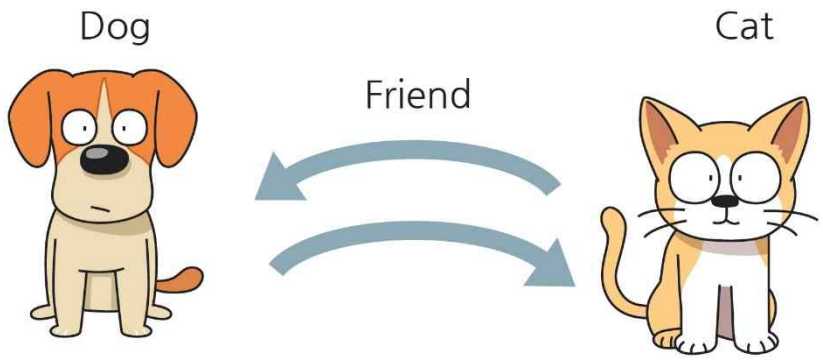
명제 논리: Kim has a house
술어 논리: HAS(Kim, house)

명제 논리: The orange is yellow
술어 논리: YELLOW(orange)

첫 번째 예에서, 'HAS'는 술어이며, 객체는 'Kim'과 'house'이다. 두 번째 예에서는 'YELLOW'는 술어이고, 'orange'는 객체이다.

객체와 관계

- 객체는 상수 기호로 나타낸다: 바둑이, 야옹이, Richard, ...
- 관계는 술어 기호로 나타낸다: HUMAN, DOG, CAT, HAT, ...



한정사

- 한정사는 변수의 범위를 서술하는 기호이다.
- 술어 논리에는 전체 한정사(universal quantifier) ∀와 존재 한정사(existential quantifier) ∃를 사용할 수 있다.
  - 전체 한정사 ∀는 "모든" 이라는 의미이다.
  - 존재 한정사 ∃는 "적어도 하나는 존재"한다는 의미이다.

All dogs like cats.



$\forall x\ [DOG(x) \rightarrow LIKES(x, cat)]$

Some dogs like cats.



$\exists x\ [DOG(x) \wedge LIKES(x, cat)]$

한정사의 순서

플레이어와 회복 아이템의 관계를 비교한다.

∀p ∃i Heals(i,p)

∃i ∀p Heals(i,p)

첫 식과 둘째 식에서 아이템 i가 같아야 할까?

게임 디자인 예를 각각 들어 보자.

질문 | 존재 한정사

적어도 한 고블린은 중립적이다.

G(x): 고블린, N(x): 중립

∃x[G(x) → N(x)]

∃x[G(x) ∧ N(x)]

어느 식이 원문을 정확히 표현하는가?

풀이 :

술어 논리의 구문

- **함수**(function)
  - 주어진 인자에 대해서 참, 거짓 값이 아닌 일반적인 값을 반환
  - 술어나 다른 함수의 인자로 사용
- **항**(term)
  - 술어의 인자가 될 수 있는 것
  - 항이 될 수 있는 것 : 개체상수, 변수, 함수

(1) 개체상수, 변수는 항이다.

(2) $t_1, t_2, \cdots, t_n$이 모두 항이고, f가 n개의 인자를 갖는 함수 기호일 때, $f(t_1, t_2, \cdots, t_n)$은 항이다.

(3) (1)과 (2)에 의해 만들어질 수 있는 것만 항이다.

술어 논리의 구문

❖ 술어 논리식에 대한 정형식

(1) $t_1, t_2, \cdots, t_n$이 모두 항이고, p가 n개의 인자를 갖는 술어 기호일 때, $p(t_1, t_2, \cdots, t_n)$은 정형식이다.

(2) p와 q가 정형식이면, 논리 기호를 사용하여 구성되는 논리식 $\neg p$, $p \vee q$, $p \wedge q$, $p \rightarrow q$, $p \equiv q$도 정형식이다.

(3) $p(x)$가 정형식이고, x가 변수일 때, $\forall x\, p(x)$, $\exists x\, p(x)$는 정형식이다.

(4) (1), (2), (3)에 의해 만들어질 수 있는 것만 술어 논리의 정형식이다.

$\forall$x $\forall$y Horse(x) $\wedge$ Dog(y) $\rightarrow$ Faster(x,y)

$\exists$y Greyhound(y) $\wedge$ ($\forall$z Rabbit(z) $\rightarrow$ Faster(y,z))

Horse(Harry)

Rabbit(Ralph)

$\forall$y Greyhound(y) $\rightarrow$ Dog(y)

$\forall$x $\forall$y $\forall$z Faster(x,y) $\wedge$ Faster(y,z) $\rightarrow$ Faster(x,z)

정형식

❖ 우리가 논리융합을 사용하려면 모든 지식이 정형식(WFF: well-formed formula)으로 표현되어야 한다.

① 기초 공식(원자)은 정형식이다.
- 그 외 동등성 술어도 정형식이다.
  - 두 변수나 함수가 같은지를 나타내는 경우
  - $x = y$, $f(x) = g(x)$, $x = f(y)$
- **논리 연산자**로 연결된 정형식이나 변수가 포함된 정형식들도 문법적 조건을 충족하는 경우에는 정형식이 될 수 있지만, 함수나 변수 그 자체만으로는 정형식이 되지 않는다.

② P와 Q가 정형식이면 $\neg P$, $P \vee Q$, $P \wedge Q$, $P \rightarrow Q$도 정형식이다.

③ P가 정형식이면 $\forall x\ P(x)$ 와 $\exists x\ P(x)$도 정형식이다.

④ 정형식은 위의 규칙을 반복하여서 형성가능하다.

정형식

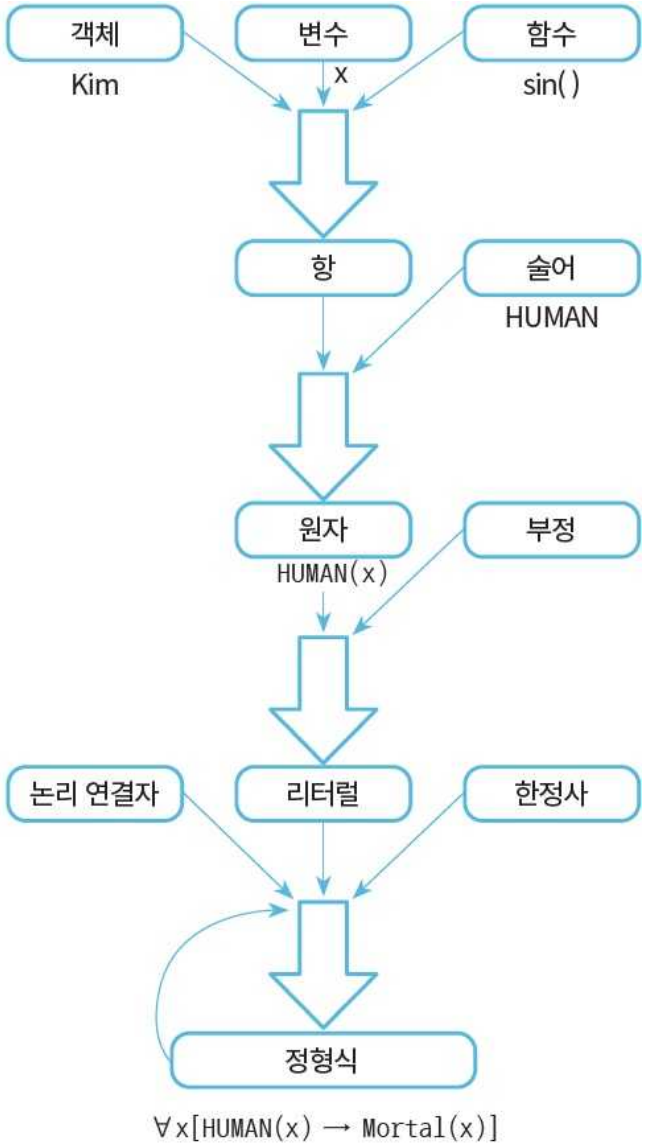


그림 5-6 정형식으로 표현하기

술어 논리의 지식표현

❖ **술어 논리를 이용한 지식 표현**

(a) Whoever can read is literate. (읽을 수 있으면 문맹이 아니다)

(b) Monkeys are not literate. (원숭이는 문맹이다)

(c) Some monkeys are intelligent. (어떤 원숭이는 지능적이다)

(d) Some who are intelligent cannot read. (지능적이어도 문맹일 수 있다) ← 증명

(a) $\forall x[CanRead(x) \rightarrow Literate(x)]$

(b) $\forall x[Monkey(x) \rightarrow \neg Literate(x)]$

(c) $\exists x[Monkey(x) \wedge Intelligent(x)]$

(d) $\exists x[Intelligent(x) \wedge \neg CanRead(x)]$

질문 | 술어논리 설계

모든 치유사는 적어도 한 플레이어를 돕는다.

Healer(x), Player(y), Helps(x,y)를 사용한다.

① 한정사 순서를 정한다.

② 플레이어마다 치유사가 달라도 되는가?

풀이 :

08 술어논리에서의 추론

- 첫 번째 방법은 술어 논리식을 명제 논리식으로 변환한 후에 명제 논리의 추론 기법을 적용하는 것
  - 도메인이 크거나 무한할 경우, 모든 경우를 나열할 수 없어서 실용성이 떨어짐
- 두번째 방법은 도출, 논리 융합(resolution)이다.

술어 논리의 추론

❖ **단일화**(unification) 과정

- 논리융합(resolution)을 적용할 때 대응되는 리터럴이 같아지도록, 변수의 값을 맞춰주는 과정

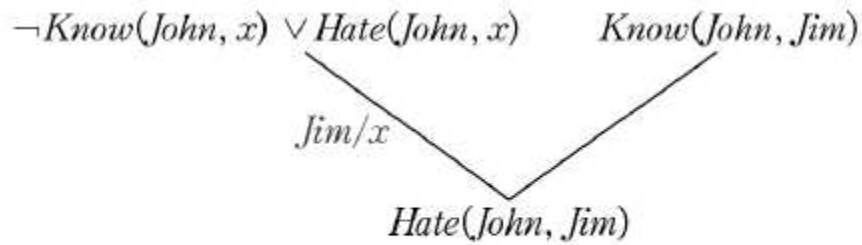


그림 3.5 단일화 과정의 예

x를 Jim으로 대체하면 논리융합에 의해 $Hate(John, Jim)$이 도출됨.

단일화 (unification)

❖ 논리융합

- **두 절이 똑같은 리터럴을 서로 상반되게 포함한 경우**

❖ 치환과 단일화의 필요성

- 술어논리에 논리융합을 적용시키기 위해 필요

  예) 다음 두 절의 논리융합을 구한다고 가정

1. $P(f(y),A) \vee Q(B,C)$
2. $\neg P(x, A) \vee R(x, C) \vee S(A, B)$
   - 두 번째 절에 치환 $\{f(y)/x\}$를 적용하면 $\neg P(f(y),A) \vee R(f(y),C) \vee S(A, B)$
   - 논리융합을 수행하면 $R(f(y),C) \vee S(A,B) \vee Q(B,C)$의 논리융합식 생성

치환(substitution)

- 정형식의 항: 변수, 객체상수, 함수식(함수기호(이름)+항)
- 정형식의 치환: 변수를 항으로 바꿈
- 순서쌍의 집합 $s = \{\tau_1/\xi_1, \tau_2/\xi_2, \ldots, \tau_n/\xi_n\}$ (τ_i: 항, ξ_i: 변수)
  - τ_i/ξ_i 의 의미: 치환이 미치는 범위 내에 있는 변수 ξ_i 모두를 항 τ_i로 바꿈
  - 어떤 식 ω를 *s*로 치환한 치환사례(substitution instance)를 ω*s*와 같이 표기

예) *s1* = {z/x, w/y} → P(x,f(y),B) *s1* = P(z, f(w), B)

s2 = {A/y} → P(x,f(y),B) *s2* = P(x, f(A), B)

s3 = {g(z)/x, A/y} → P(x,f(y),B) *s3* = P(g(z), f(A), B)

s4 = {C/x, A/y} → P(x,f(y),B) *s4* = P(C, f(A), B)

단일화 (unification)

❖ 단일화가 가능하다(unifiable)

- $\omega_1 s = \omega_2 s = \omega_3 s = \ldots$의 관계가 성립하는 치환 s가 존재할 때 - 치환 s를 $\{\omega_i\}$의 **단일자**(unifier)라고 함

  예) {P[x,f(y),B], P[x,f(B),B]}에 대한 단일자

  $s1$ = {A/x, B/y}, $s2$ = {B/y}

- **가장 일반적인 단일자**(most general unifier, *mgu*) ***g***
  - 치환 s에 의해 정형식의 집합 $\{\omega_i\}$가 $\{\omega_i\}s$로 단일화된다면 $\{\omega_i\}s = \{\omega_i\}gs'$의 관계가 성립하는 치환 s'이 존재한다.
    - s는 두 식을 같게 만드는 **임의의 단일자**, s'는 mgu g의 결과를 더 구체적인 값으로 바꾸는 **추가 치환**
  - *mgu*에 의한 결과는 알파벳 변이를 제외하고는 유일
  - UNIFY 알고리즘: *mgu*를 구하는 알고리즘 (불일치 집합의 개념)

질문 | 단일화 가능성

전투 기록 두 개를 같은 사실로 만들 수 있을까?

Attack(x,x) 와 Attack(hero,boss)

Attack(x,y) 와 Attack(hero,boss)

각각 단일화 가능 여부와 가장 일반적인 치환을 적자.

풀이 :

논리융합을 사용하려면

- 논리융합 기법으로 증명하려면 모든 논리식들을 논리곱 표준형(CNF: conjunctive normal form)으로 바꾸어야 한다.
- 모든 전체 한정사는 없애버린다(기본적으로 가정된다).
- 존재 한정사는 스콜렘 함수로 바꾼다.

술어 논리의 추론

❖ **술어 논리식의 CNF로의 변환 과정**

1. 전체 한정사와 존재 한정사를 논리식의 맨 앞으로 끌어내는 변환

2. **전체 한정사**에 결합된 변수
   - 임의의 값 허용
3. **존재 한정사**에 결합된 변수
   - 대응되는 술어 기호를 참(T)으로 만드는 값을 변수에 대응시킴
   - **스콜렘 함수(Skolem function)**
     - 존재 한정사에 결합된 변수를 해당 술어의 **전체 한정사**에 결합된 다른 **변수**들의 새로운 **함수로 대체**
     - $\forall x \exists y [P(x) \wedge Q(x, y)] \Rightarrow \forall x [P(x) \wedge Q(x, s(x))]$
     - $s(x)$
       - » $Q(x, s(x))$를 어떤 x에 대해서도 참으로 만드는 마법의 함수(magic function)

스콜렘 함수의 역할

증인의 값이 플레이어에 의존한다.

∀p ∃i Owns(p,i)

스콜렘화: Owns(p, itemFor(p))

itemFor(p)는 코드에서 실제 아이템을 계산하는 함수인가?

임의의 정형식을 절 형태로 변경하기

❖ **절 형태(clausal form)로의 변경 단계**

1. 함의(implication) 부호를 제거
2. 이중 부정 제거
3. 변수표준화: 각 한정사가 한정하는 변수의 이름이 유일하도록 변수명을 바꿈
4. 존재한정사를 제거한다. (Skolem 함수이용)
5. prenex 형식으로 변경 : 모든 전체한정사를 정형식의 제일 앞으로 이동
6. CNF 정규형으로 변형
7. 전체한정사를 제거
8. 논리곱(∧) 심볼을 제거 : $(\omega_1 \wedge \omega_2)$ 형식의 식을 정형식의 집합 $\{\omega_1, \omega_2\}$로 대치
9. 절마다 변수명을 변경

질문 | 존재 변수 제거

다음 식을 스콜렘화하자.

∀n ∃w HasWeapon(n,w)

① 존재 변수 w를 무엇으로 바꿀까?

② ∃w HasWeapon(boss,w)라면 무엇으로 바꿀까?

풀이 :

술어논리 논리융합

❖ Resolution in predicate logic

- γ_1과 γ_2: 리터럴의 집합으로 표현된 절
  γ_1은 아톰 ϕ 포함, γ_2는 리터럴 $\neg\psi$ 포함
  ϕ 와 ψ 가 mgu μ를 가지고 있다면
  - 이 두 절은 다음과 같은 논리융합식 ρ를 가짐
    $\rho = [(\gamma_1 - \{\phi\}) \cup (\gamma_2 - \{\neg\psi\})]\mu$

예) 다음 두 절을 논리융합하려면

- $P(x) \vee Q(f(x))$
- $R(g(x)) \vee \neg Q(f(A))$
- 먼저 두 번째 절을 $R(g(y)) \vee \neg Q(f(A))$로 변경
- 논리융합을 수행하면 $P(A) \vee R(g(y))$를 얻음

❖ 예2

- {P(x),Q(x,y)}와 {¬P(A),R(B,z)}를 논리융합하면 {Q(A,y),R(B,z)}

❖ 예3

- {P(x,x),Q(x),R(x)}와 {¬P(A,z),¬Q(B)}를 논리융합하면
- {Q(A),R(A),¬Q(B)} 또는 {P(B,B),R(B),¬P(A,z)}

술어 논리의 추론

❖ 술어 논리로 지식의 증명

(a) $\forall x[CanRead(x) \rightarrow Literate(x)]$

(b) $\forall x[Monkey(x) \rightarrow \neg Literate(x)]$

(c) $\exists x[Monkey(x) \wedge Intelligent(x)]$

(d) $\exists x[Intelligent(x) \wedge \neg CanRead(x)]$ ← **증명**

- 논리합 형태로 변환

(1) $\neg CanRead(x) \vee Literate(x)$

(2) $\neg Monkey(x) \vee \neg Literate(x)$ x를 y로변경

(3) $Monkey(A)$

(4) $Intelligent(A)$

(5) $\neg Intelligent(x) \vee CanRead(x)$ x를 z로변경

(d)를 **논리융합 반박**을 이용하여 증명하기 부정

- (1)과 (5)

(6) $\neg Intelligent(x) \vee Literate(x)$

- (2)와 (6)

(7) $\neg Intelligent(x) \vee \neg Monkey(x)$

- (3)과 (7)

(8) $\neg Intelligent(A)$

- (4)와 (8)

(9) $Intelligent(A) \wedge \neg Intelligent(A) \equiv nil$

(d)를 부정하여 모순이 발생하므로, (d)가 참임

술어 논리의 추론

❖ 술어 논리로 지식의 증명

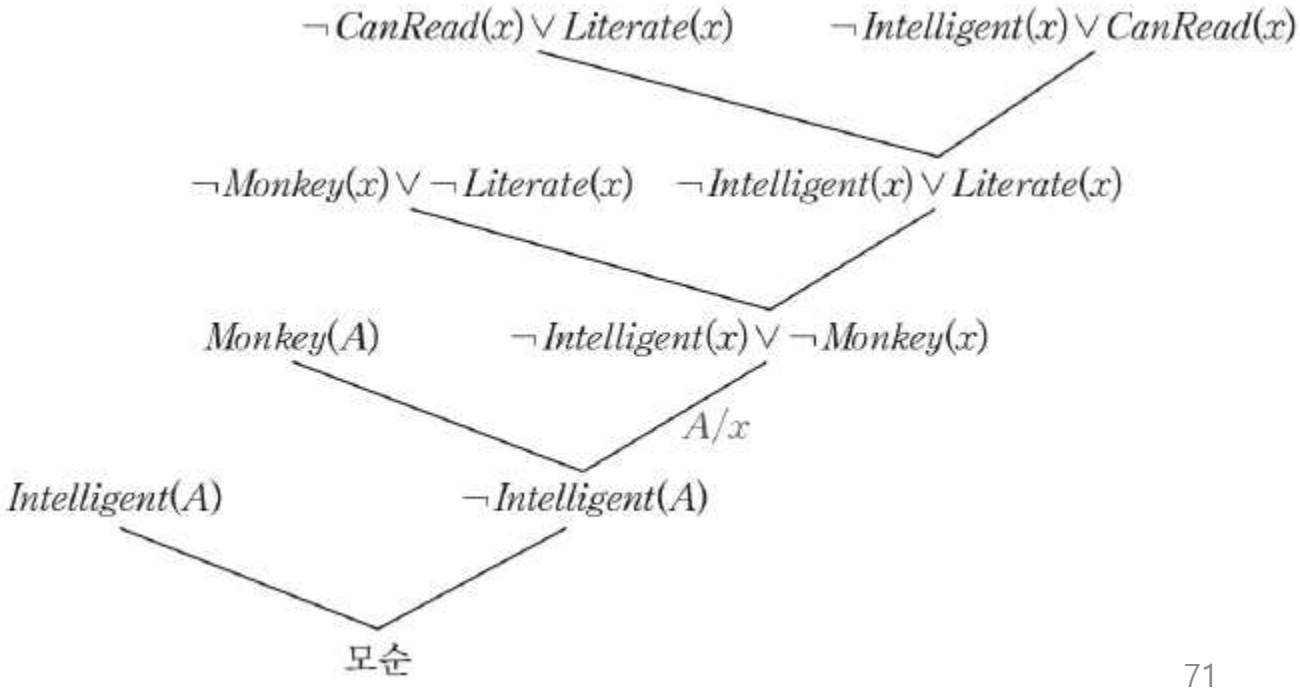
(1) $\neg CanRead(x) \vee Literate(x)$

(2) $\neg Monkey(x) \vee \neg Literate(x)$

(3) $Monkey(A)$

(4) $Intelligent(A)$

(5) $\neg Intelligent(x) \vee CanRead(x)$



논리융합을 이용한 정리증명

❖ 정리증명 단계

- Δ에 있는 모든 정형식을 절 형태로 변형
- 증명하고자 하는 정형식 ω를 부정(negate)한 절을 Δ에 대한 절에 추가
- 공절이 추론될 때까지 논리융합을 적용

❖ 예제

- "모든 사람은 죽는다"와 "소크라테스는 사람이다"로부터 "소크라테스는 죽는다"를 추론
- $\Delta = \{(\forall x)[Man(x) \supset Mortal(x)], Man(Socrates)\}$
- $\omega = Mortal(Socrates)$

1. 절 형태로 변형 : {¬Man(x)∨ Mortal(x), Man(Socrates)}
2. ω의 부정을 추가: ¬Mortal(Socrates)
3. 논리융합을 적용
    1. ¬Man(x)∨ Mortal(x)
    2. Man(Socrates)
    3. ¬Mortal(Socrates)
    4. ¬Man(Socrates) (1,3을 논리융합 - *mgu* = {Socrates/x})
    5. NIL (2,4를 논리융합)

질문 | 반박 증명

보스가 취약 대상임을 증명하자.

$\forall x[\text{IceWeak}(x) \rightarrow \text{Vulnerable}(x)]$

IceWeak(boss)

목표: Vulnerable(boss)

목표 부정을 추가한 뒤 공절까지 유도한다.

풀이 :

논리융합에 의한 증명

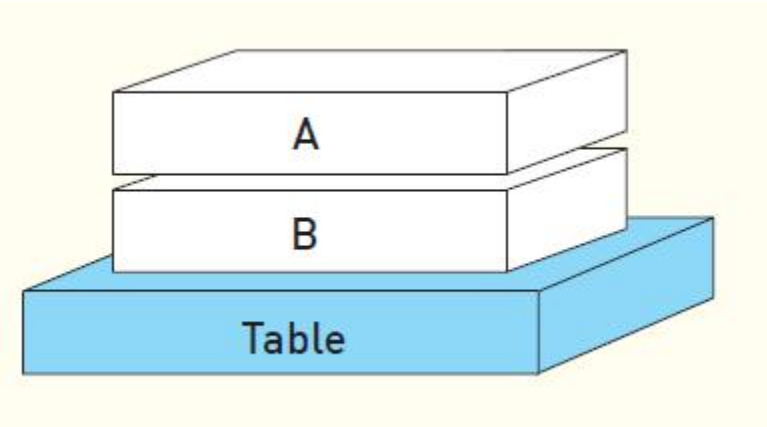
1. 증명하고자 하는 사실을 부정하여 절들의 리스트에 추가한다.
2. 지식 베이스의 문장들을 CNF 형태로 변환한다.
3. 논리융합할 수 있는 절의 쌍이 더 이상 없을 때까지 다음을 반복한다.
    3.1. 논리융합할 수 있는 절의 쌍을 찾아 논리융합한다.
    3.2. 논리융합절을 절들의 리스트에 추가한다.
    3.3. NIL이 유도되면, 증명하고자 하는 사실이 참이다.
4. 증명하고자 하는 사실의 부정이 거짓이다.

Lab: 논리융합 연습 #1

- 간단한 논리융합 연습을 해보자. 다음과 같은 지식이 있다.
  ① 어떤 것이 지능적이라면 그것은 상식을 가져야 한다.
  ② PC는 상식이 없다.
- 위의 지식을 바탕으로 'PC는 지능적이 아니다'를 증명해 보자.

Lab: 논리융합 연습 #3

- 다음과 같은 상황을 술어 논리를 이용하여 표현하고 ABOVE(A, Table)을 증명해 보자.



Lab: 논리융합 연습 #3

❖ 테이블 위에 블록 B가 놓여져 있고 블록 B 위에 블록 A가 놓여져 있다. 따라서 우리가 관찰한 사실은 다음과 같이 술어 논리로 표현할 수 있다.

ON(A, B)

ON(B, Table)

❖ 우리는 상식으로 다음과 같은 사실을 알고 있다.

$\forall x \forall y$ [ON(x, y) $\rightarrow$ ABOVE(x, y)]

$\forall x \forall y \forall z$ [ABOVE(x, y) $\wedge$ ABOVE(y, z) $\rightarrow$ ABOVE(x, z)]

논리 프로그래밍 언어

- **Horn 절 (Horn clause)**
  - 논리식을 논리합의 형태로 표현할 때, $\neg A(x) \vee \neg B(x) \vee C(x)$ 와 같이 긍정인 리터럴을 최대 하나만 허용
- **Prolog**
  - Prolog의 기본 규칙은 definite Horn 절에 대응한다.
    - Horn 절은 긍정 리터럴이 최대 하나, definite Horn 절은 정확히 하나
    - 규칙 본문의 쉼표는 논리곱이며, 변수는 대문자로 시작한다.

```
father(noah, shem).
father(noah, ham).
father(shem, elam).
father(shem, arphaxad).
father(arphaxad, caina).
grandfather(X,Y) :- father(X,Z), father(Z,Y).
?- grandfather(noah, Y).
```

father(X,Z) ∧ father(Z,Y) → grandfather(X,Y)

    - Y 값 출력
  - 백트랙킹(backtracking)을 이용하여 실행

프롤로그(Prolog)

- https://www.swi-prolog.org/에서 다운로드할 수 있는 SWI-Prolog를 설치해보자.

```
parent(Y,X) :- child(X,Y).
father(Y,X) :- child(X,Y), male(Y).
```

- 프롤로그에서는 변수를 항상 대문자로 작성해야 한다.
- "father(Y,X) :- child(X,Y), male(Y)."은 X가 Y의 자식이고 Y가 남성이면 Y는 X의 아버지이다. 를 의미한다. 즉 child(X,Y) ∧ male(Y) → father(Y,X)을 의미한다.
- 규칙 본문의 쉼표는 논리곱을 뜻하며 ;로 대안도 표현할 수 있다.

프롤로그(Prolog)에서 사실 입력

❖ 상수는 소문자로 입력한다.

```
child(son1, kim).
child(son2, kim).
male(kim).
male(son1).
male(son2).
```

프롤로그 실습

① 다음과 같은 문장들을 저장하는 파일 main.pl을 생성한다.

```
child(son1, kim).
child(son2, kim).
male(kim).
male(son1).
male(son2).
parent(Y,X) :- child(X,Y).
father(Y,X) :- child(X,Y), male(Y).
```

② [File]->[Consult...] 메뉴 항목을 선택하여서 main.pl을 불러들인다.

질문 | Prolog 질의

NPC의 위협 관계를 질의해 보자.

```
sees(guard,hero).
armed(guard).
threatens(X,Y) :- sees(X,Y), armed(X).
?- threatens(guard,Who). 의 결과는?
```

풀이 :

Summary

- 명제논리는 사실의 참·거짓과 연결 관계를 표현한다.
- 술어논리는 개체, 관계, 변수, 한정사를 추가한다.
- CNF와 단일화는 논리융합을 준비하는 핵심 단계다.
- Prolog는 사실·Horn 절을 바탕으로 질의한다.

Q & A

